Lý thuyết cần thiết trước khi làm đồ án

So sánh hiệu năng CPU, GPU và Systolic Array trên bài toán nhân ma trận

Nhóm đồ án MI4344 — FaMI-HUST 2026

Tài liệu nội bộ (nhập môn)

Abstract

Tài liệu này giải thích *bằng ngôn ngữ dễ hiểu* các khái niệm kiến trúc máy tính mà nhóm cần nắm trước khi viết code và đo hiệu năng. Mục tiêu không phải thay sách giáo khoa, mà giúp sinh viên trả lời được: *đang đo cái gì, vì sao số liệu tăng/giảm, và so sánh thế nào cho công bằng*. Công thức được viết đầy đủ bằng LaTeX để dùng lại trong báo cáo.

Contents

1 Cách dùng tài liệu này **3**
- 1.1 Đọc theo lộ trình . . . 3
- 1.2 Phạm vi kỹ thuật đã chốt của đồ án . . . 3

2 Nhân ma trận và đơn vị FLOPs **3**
- 2.1 Định nghĩa bài toán . . . 3
- 2.2 Đếm số phép toán: vì sao là $2N^3$? . . . 3
- 2.3 Công thức GFLOPS dùng khi đo CPU/GPU . . . 4
- 2.4 Vì sao AI "sống nhờ" nhân ma trận? . . . 4

3 Ba nút thắt hiệu năng **4**
- 3.1 Compute-bound (thiếu đơn vị tính) . . . 4
- 3.2 Memory-bound (chờ dữ liệu) . . . 5
- 3.3 Latency / overhead (chi phí cố định) . . . 5

4 Bộ nhớ, locality và cache tiling **5**
- 4.1 Tháp bộ nhớ (đơn giản hóa) . . . 5
- 4.2 Hai loại locality . . . 5
- 4.3 Cache blocking (tiling) . . . 5

5 Song song trên CPU: đa lõi và SIMD **6**
- 5.1 Đa lõi (OpenMP) . . . 6
- 5.2 SIMD (Single Instruction, Multiple Data) . . . 6
- 5.3 Ước lượng peak GFLOPS của CPU . . . 6
- 5.4 Speedup và ablation . . . 7

6 GPU: thông lượng cao và chi phí copy **7**
- 6.1 Triết lý thiết kế . . . 7
- 6.2 Luồng dữ liệu Host – Device . . . 7
- 6.3 Điểm hòa vốn $N^\star$ . . . 7
- 6.4 Kernel tự viết và cuBLAS . . . 8

7 Metric hiệu năng — nói cùng một ngôn ngữ **8**
7.1 Utilization cho systolic array . . . 8
7.2 Quy tắc vàng của đồ án . . . 8

8 Systolic array **8**
8.1 Ý tưởng cốt lõi . . . 8
8.2 Dataflow weight-stationary (đã chốt) . . . 9
8.3 Kiểu số: int16 trên RTL, float32 trên CPU/GPU . . . 9
8.4 Lưới 8×8 và tiling . . . 9
8.5 Throughput lý thuyết khi đầy tải . . . 9
8.6 Mô hình chu kỳ (khung để hoàn thiện khi chốt TB) . . . 9

9 Ba kiến trúc — bảng triết lý **10**

10 Verilog và mô phỏng — vừa đủ để bắt đầu **10**
10.1 HDL khác chương trình phần mềm thế nào? . . . 10
10.2 Thuật ngữ tối thiểu . . . 10
10.3 Chu trình làm việc Module 3 (toàn bộ do P4) . . . 10

11 Bốn câu hỏi nghiên cứu gắn với lý thuyết **10**

12 Lộ trình đọc theo vai **11**

13 Bảng thuật ngữ (glossary) **11**

14 Sai lầm thường gặp **11**

15 Tài liệu tham khảo gợi ý **11**

1. Cách dùng tài liệu này

1.1. Đọc theo lộ trình

1. **Trước Tuần 1 (cả nhóm):** mục 2 → 3 → 4 → 7 → 9.
2. **Theo vai:**
   - P1–P2 (CPU): thêm mục 5.
   - P3 (GPU): thêm mục 6.
   - P4 (RTL + TB + sim): thêm mục 8 và 10.
   - P5 (so sánh Câu 4): đọc kỹ mục 7 và 9.
3. Sau khi đọc: mỗi người viết 5 dòng *"Module của tôi đang giảm nút thắt nào?"* mang vào họp nhóm.

1.2. Phạm vi kỹ thuật đã chốt của đồ án

Thành phần	Quyết định
CPU / GPU	số thực `float32`
Systolic array	lưới PE 8×8, dataflow *weight-stationary*, MAC `int16`
Mô phỏng HDL	Icarus Verilog + GTKWave
So sánh chéo	*không* xếp hạng mili-giây tuyệt đối cả ba nền

2. Nhân ma trận và đơn vị FLOPs

2.1. Định nghĩa bài toán

Cho hai ma trận vuông $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{N \times N}$. Tích $\mathbf{C} = \mathbf{AB}$ được định nghĩa phần tử theo phần tử:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik}\, B_{kj}, \qquad i, j \in \{0, 1, \ldots, N-1\}. \tag{1}$$

Trong lập trình, công thức (1) thường hiện thành ba vòng lặp lồng nhau theo chỉ số (i, j, k) (thứ tự vòng lặp có thể đổi, nhưng số phép tính không đổi).

2.2. Đếm số phép toán: vì sao là $2N^3$?

Với mỗi cặp (i, j):

- cần N phép **nhân** $A_{ik} \cdot B_{kj}$;
- cần khoảng N phép **cộng** để cộng dồn thành C_{ij}.

Có N^2 phần tử của $\mathbf{C}$, nên tổng số phép toán số học (FLOPs) là

$$\text{FLOPs}(N) = 2N^3. \tag{2}$$

Nếu phần cứng dùng lệnh FMA (*Fused Multiply-Add*)

$$d \leftarrow a \cdot b + c,$$

mỗi FMA vẫn được quy ước đếm **2 FLOPs** (một nhân + một cộng) khi báo cáo GFLOPS trong cộng đồng HPC/AI — nhờ đó công thức (2) vẫn dùng được.

Ví dụ số

Với $N = 1024$:

$$\text{FLOPs} = 2 \cdot (1024)^3 = 2 \cdot 1\,073\,741\,824 = 2\,147\,483\,648 \approx 2.15 \times 10^9.$$

Nếu chương trình chạy mất $t = 0.50\,\text{s}$ thì

$$\text{GFLOPS} = \frac{2N^3}{t \cdot 10^9} = \frac{2.147 \times 10^9}{0.50 \cdot 10^9} \approx 4.3.$$

2.3. Công thức GFLOPS dùng khi đo CPU/GPU

$$\text{GFLOPS} = \frac{2N^3}{t_{\text{sec}} \cdot 10^9} = \frac{2N^3}{t_{\text{ms}} \cdot 10^6}, \tag{3}$$

trong đó t_{sec} (hoặc t_{ms}) là thời gian tường (*wall-clock*) đo được cho một lần nhân ma trận.

Áp dụng vào đồ án

Mọi bản code CPU/GPU của nhóm (naive, tiled, OpenMP, CUDA, cuBLAS) đều dùng cùng (2)–(3). Như vậy khi thời gian giảm, GFLOPS tăng — *không phải vì thuật toán tính ít phép hơn*, mà vì phần cứng khai thác song song / bộ nhớ tốt hơn.

2.4. Vì sao AI "sống nhờ" nhân ma trận?

- **Fully-connected:** $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ là nhân ma trận–vector (hoặc ma trận–ma trận khi gộp nhiều mẫu thành batch).
- **Convolution:** có thể rearrange thành GEMM (kỹ thuật *im2col* / *im2row*).
- **Attention** (Transformer): cần $\mathbf{QK}^\mathsf{T}$ rồi nhân với $\mathbf{V}$ — lại là GEMM.

Vì vậy nhân ma trận là *micro-benchmark* lý tưởng: thuật toán đơn giản, nhưng hiệu năng phụ thuộc cực mạnh vào kiến trúc.

3. Ba nút thắt hiệu năng

Trên mọi máy tính, tốc độ thực tế bị giới hạn bởi ít nhất một trong các nút sau.

3.1. Compute-bound (thiếu đơn vị tính)

Máy "đói" phép nhân/cộng: ALU/FPU/PE chạy gần tối đa, tăng thêm đơn vị tính sẽ giúp rõ. Khi đó thời gian xấp xỉ

$$t \approx \frac{\text{FLOPs}}{\text{PeakOpsPerSecond} \cdot U}, \tag{4}$$

với $U \in (0, 1]$ là hệ số tận dụng (*utilization*).

3.2. Memory-bound (chờ dữ liệu)

Đơn vị tính rảnh vì phải chờ cache/RAM/HBM. Băng thông bộ nhớ B (byte/s) và cường độ số học

$$I = \frac{\text{số FLOPs}}{\text{số byte đọc/ghi từ bộ nhớ ngoài}} \quad [\text{ops/byte}] \tag{5}$$

quyết định trần Roofline:

$$\text{GFLOPS}_{\text{attainable}} = \min\big(\text{Peak}_{\text{compute}},\ I \cdot B_{\text{GB/s}}\big). \tag{6}$$

3.3. Latency / overhead (chi phí cố định)

Khởi động kernel GPU, copy Host↔Device, tạo thread, đồng bộ…tốn một khoảng thời gian gần như không phụ thuộc N (hoặc tăng chậm). Với N nhỏ, overhead chiếm tỷ trọng lớn ⇒ GPU có thể *thua* CPU dù peak GFLOPS của GPU cao hơn rất nhiều.

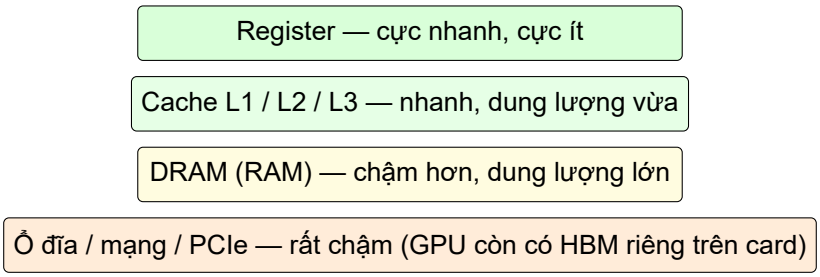
Lưu ý quan trọng

Khi viết báo cáo, đừng chỉ nói "nhanh hơn X lần". Hãy chỉ ra đang giảm nút thắt nào: cache miss, thiếu SIMD, overhead copy, …

---

4. Bộ nhớ, locality và cache tiling

4.1. Tháp bộ nhớ (đơn giản hóa)



CPU *không* lấy từng số `float` từ RAM mỗi lần nhân. Nó đọc theo **dòng cache** (*cache line*), thường 64 byte (= 16 số `float32`). Nếu vòng lặp nhảy lung tung trên ma trận lớn ⇒ nhiều *cache miss* ⇒ chậm dù số FLOPs không đổi.

4.2. Hai loại locality

Temporal locality
: Dùng lại cùng một phần tử trong thời gian ngắn. Ví dụ: A_{ik} được dùng cho nhiều cột j.

Spatial locality
: Dùng các phần tử nằm gần nhau trên bộ nhớ. Với lưu trữ *row-major*, duyệt theo hàng của **A** tốt hơn duyệt theo cột của **B**.

4.3. Cache blocking (tiling)

Ý tưởng: chia bài toán $N \times N$ thành các khối kích thước $T \times T$ vừa với cache, tính hết một khối trước khi chuyển sang khối khác.

Một dạng giả mã:

$$\begin{aligned}&\textbf{for } ii = 0, T, 2T, \dots \\ &\quad \textbf{for } jj = 0, T, 2T, \dots \\ &\quad\quad \textbf{for } kk = 0, T, 2T, \dots \\ &\quad\quad\quad \text{nhân khối } \mathbf{A}[ii : ii+T,\ kk : kk+T] \text{ với } \mathbf{B}[kk : kk+T,\ jj : jj+T] \\ &\quad\quad\quad \text{cộng vào } \mathbf{C}[ii : ii+T,\ jj : jj+T].\end{aligned} \tag{7}$$

Áp dụng vào đồ án

So sánh `naive` và `tiled` trên cùng N: FLOPs vẫn là $2N^3$ (2). Nếu thời gian giảm, nguyên nhân chính là **ít cache miss hơn**, không phải "tính ít đi". Đây là trụ cột trả lời **Câu hỏi nghiên cứu 1**.

---

5. Song song trên CPU: đa lõi và SIMD

5.1. Đa lõi (OpenMP)

Máy có P lõi có thể chạy song song nhiều hàng i (hoặc nhiều tile). Speedup lý tưởng theo Amdahl bị giới hạn bởi phần tuần tự s:

$$S(P) = \frac{1}{s + (1-s)/P} \le \frac{1}{s}. \tag{8}$$

Thực tế còn bị tranh chấp bandwidth bộ nhớ dùng chung và chi phí đồng bộ.

5.2. SIMD (Single Instruction, Multiple Data)

Một lệnh máy tác động đồng thời lên L phần tử (ví dụ AVX2 với `float32`: $L = 8$). Hình ảnh trực giác:

$$\begin{aligned}&\text{Scalar:} \quad a_0 \cdot b_0, \\ &\text{SIMD:} \quad (a_0, a_1, \dots, a_{L-1}) \odot (b_0, b_1, \dots, b_{L-1}) \quad \text{trong một nhịp lệnh.}\end{aligned}$$

5.3. Ước lượng peak GFLOPS của CPU

Một ước lượng thô (chỉ để định hướng, không cần chính xác tuyệt đối):

$$\text{Peak}_{\text{CPU}} \approx P \cdot L \cdot F \cdot f_{\text{GHz}} \cdot 2 \quad [\text{GFLOPS}], \tag{9}$$

trong đó

- P: số lõi được dùng,
- L: số lane SIMD (`float32`),
- F: số FMA mỗi chu kỳ mỗi lõi (thường 1 hoặc 2),
- hệ số 2: mỗi FMA = 2 FLOPs,
- f_{GHz}: tần số clock (GHz).

Hiệu năng *sustained* thực tế thường chỉ đạt một phần của peak — điều này là bình thường.

5.4. Speedup và ablation

Speedup của bản tối ưu so với baseline:

$$S = \frac{T_{\text{baseline}}}{T_{\text{new}}}. \tag{10}$$

Áp dụng vào đồ án

Ablation đúng nghĩa: thêm **một** kỹ thuật mỗi lần

`naive` $\rightarrow$ `tiled` $\rightarrow$ `+OpenMP` $\rightarrow$ `+SIMD/BLAS`,

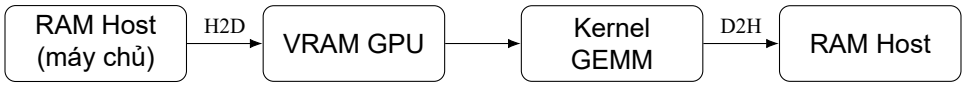
đo S theo (10) trên cùng N. Tránh bật tất cả flag cùng lúc rồi không giải thích được đóng góp từng phần (Câu 1).

6. GPU: thông lượng cao và chi phí copy

6.1. Triết lý thiết kế

CPU	GPU
Ít lõi mạnh, cache lớn, giỏi độ trễ và nhánh phức tạp	Hàng nghìn thread nhẹ; giấu độ trễ bộ nhớ bằng cách chuyển sang thread khác sẵn sàng

6.2. Luồng dữ liệu Host – Device



Hai cách đo thời gian *bắt buộc* trong đồ án:

$$T_{\text{kernel}} = \text{chỉ thời gian chạy kernel trên GPU}, \tag{11}$$

$$T_{\text{E2E}} = T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}}. \tag{12}$$

GFLOPS tương ứng:

$$\text{GFLOPS}_{\text{kernel}} = \frac{2N^3}{T_{\text{kernel}} \cdot 10^9}, \tag{13}$$

$$\text{GFLOPS}_{\text{E2E}} = \frac{2N^3}{T_{\text{E2E}} \cdot 10^9}. \tag{14}$$

6.3. Điểm hòa vốn $N^\star$

Gọi $T^\star_{\text{CPU}}(N)$ là thời gian CPU tốt nhất của nhóm tại kích thước N. Định nghĩa

$$N^\star_{\text{E2E}} = \min\{N \mid T^{\text{GPU}}_{\text{E2E}}(N) < T^\star_{\text{CPU}}(N)\}. \tag{15}$$

Tương tự có thể định nghĩa $N^\star_{\text{kernel}}$ khi chỉ so T_{kernel}.

Lưu ý quan trọng

$N^\star$ **phụ thuộc máy**: Colab có thể cấp T4 hoặc L4, băng thông khác nhau. Luôn ghi model GPU, CUDA, và báo *khoảng* hòa vốn thay vì một số "thần thánh" duy nhất.

6.4. Kernel tự viết và cuBLAS

- Kernel naive: mỗi thread tính một C_{ij} theo (1) — dễ hiểu, thường chậm.
- **cuBLAS**: thư viện GEMM tối ưu của NVIDIA — cận trên thực tế trên GPU đó.

Áp dụng vào đồ án

Trả lời **Câu 2**: vẽ $T^{\star}_{\text{CPU}}(N)$, $T_{\text{kernel}}(N)$, $T_{\text{E2E}}(N)$; ước lượng $N^{\star}_{\text{E2E}}$ và $N^{\star}_{\text{kernel}}$ theo (15).

---

7. Metric hiệu năng — nói cùng một ngôn ngữ

Metric	Định nghĩa	Dùng khi
Latency	Thời gian một lần chạy T (ms)	So CPU $\leftrightarrow$ GPU cùng N
Throughput	Việc làm được mỗi giây (GFLOPS, tile/s)	Khả năng "cày" dữ liệu lớn
Speedup S	$T_{\text{old}}/T_{\text{new}}$ — xem (10)	Ablation CPU
Utilization U	Tỷ lệ phần cứng làm việc hữu ích	Systolic; đôi khi GPU
Peak	Trần lý thuyết phần cứng	Báo "đạt bao nhiêu % peak"

7.1. Utilization cho systolic array

Với lưới có P_{PE} phần tử xử lý, trong C chu kỳ clock, số MAC hữu ích là M_{useful}. Peak MAC trong khoảng đó là $P_{\text{PE}} \cdot C$.

$$U = \frac{M_{\text{useful}}}{P_{\text{PE}} \cdot C} \in [0, 1]. \tag{16}$$

7.2. Quy tắc vàng của đồ án

1. **CPU/GPU:** báo T (ms thật) và GFLOPS theo (3).
2. **Systolic (mô phỏng):** báo số chu kỳ C, utilization U (16); *không* đổi chu kỳ sang ms rồi tuyên bố "nhanh hơn RTX", trừ khi nêu rõ giả định tần số f và chỉ dùng để thảo luận lý thuyết.
3. **Câu 4:** bảng đánh đổi (linh hoạt / thông lượng / chi phí bộ nhớ / độ khó thiết kế), không phải bảng xếp hạng tuyệt đối.

8. Systolic array

8.1. Ý tưởng cốt lõi

Systolic lấy cảm hứng từ nhịp đập (systole): dữ liệu chảy có nhịp qua lưới các **PE** (*processing element*) giống nhau. Mỗi PE gần như chỉ làm:

$$\text{acc} \leftarrow \text{acc} + a \cdot w \qquad \text{(MAC: Multiply-Accumulate)}. \tag{17}$$

Sau đó chuyển a và/hoặc partial sum sang hàng xóm. Lợi ích lớn: **tái sử dụng dữ liệu trên lưới**, giảm số lần đọc lại từ bộ nhớ ngoài.

8.2. Dataflow weight-stationary (đã chốt)

Tín hiệu	Hành vi
Weight w	Nạp vào PE và **đứng yên**
Activation a	Chảy theo một hướng (thường trái $\rightarrow$ phải)
Partial sum	Chảy theo hướng kia (thường trên $\rightarrow$ dưới)

Đây là tinh thần dataflow gần với Google TPU (nhóm đang làm bản rút gọn học thuật).

8.3. Kiểu số: int16 trên RTL, float32 trên CPU/GPU

Trên RTL đồ án:

$$w, a \in \mathbb{Z} \cap [-2^{15}, 2^{15}-1] \quad (\texttt{int16}), \qquad \text{acc} \in \texttt{int32} \tag{18}$$

để giảm tràn khi cộng dồn. CPU/GPU đo bằng `float32`. Hai thế giới **không cùng một "GFLOPS"** — báo cáo phải nói rõ sự khác biệt.

8.4. Lưới 8×8 và tiling

Số PE:

$$P_{\text{PE}} = 8 \times 8 = 64. \tag{19}$$

Ma trận $N > 8$ được cắt thành các tile 8×8 và chạy lần lượt. Số tile theo mỗi chiều khoảng $\lceil N/8 \rceil$, nên số tile ma trận khoảng

$$N_{\text{tile}} \approx \left\lceil \frac{N}{8} \right\rceil^2 \quad \text{(ước lượng thô; chi tiết phụ thuộc lịch trình cụ thể).} \tag{20}$$

8.5. Throughput lý thuyết khi đầy tải

Mỗi chu kỳ, tối đa khoảng P_{PE} phép MAC. Nếu mỗi MAC đếm 2 ops:

$$\text{ops/cycle}_{\text{peak}} \approx 2 \cdot P_{\text{PE}} = 128. \tag{21}$$

Nếu giả định tần số f (Hz) chỉ để thảo luận:

$$\text{Throughput}_{\text{peak}} \approx 2 \cdot P_{\text{PE}} \cdot f \quad [\text{ops/s}]. \tag{22}$$

Thực tế $U < 1$ vì pha nạp weight, pipeline fill/drain, biên tile.

8.6. Mô hình chu kỳ (khung để hoàn thiện khi chốt TB)

Một dạng tổng quát nhóm sẽ khớp với waveform:

$$C(N) = C_{\text{load}} + C_{\text{compute}}(N) + C_{\text{drain}}, \tag{23}$$

trong đó $C_{\text{compute}}(N)$ tăng theo số tile và độ sâu pipeline. **Không cần** mô phỏng đầy đủ RTL với $N = 4096$; hãy mô phỏng vài N nhỏ/tile và dùng (23) để ngoại suy có kiểm chứng.

Áp dụng vào đồ án

Trả lời **Câu 3:** (1) giải thích data reuse trên lưới; (2) đo/ước lượng $C(N)$ và U; (3) đối chiếu GTKWave với công thức.

---

9. Ba kiến trúc — bảng triết lý

	CPU	GPU	Systolic
Triết lý	Ít lõi mạnh, đa năng	Rất nhiều thread đơn giản	Phần cứng làm đúng một kiểu tính
Điểm mạnh	Linh hoạt, độ trễ thấp	Thông lượng cao khi N lớn	Hiệu suất MAC/area, data reuse
Điểm yếu	Ít đơn vị tính	Overhead copy/kernel	Không linh hoạt; RTL khó
Cách nhóm đo	Laptop thật (ms, GFLOPS)	Colab CUDA (ms, GFLOPS)	Verilog: chu kỳ, U
Ví dụ đời thực	Core / Ryzen	RTX, Tesla	TPU, NPU điện thoại

Đây là xương sống cho **Câu hỏi 4**: đánh đổi linh hoạt $\leftrightarrow$ hiệu suất.

10. Verilog và mô phỏng — vừa đủ để bắt đầu

10.1. HDL khác chương trình phần mềm thế nào?

HDL (Verilog/VHDL) *mô tả phần cứng*: thanh ghi, dây nối, cạnh clock. Nhiều khối chạy **song song theo nhịp clock**, không phải một luồng Python tuần tự.

10.2. Thuật ngữ tối thiểu

Khái niệm	Ý nghĩa
`clk`	Xung đồng hồ; cập nhật thanh ghi ở cạnh lên
`reg` / `wire`	Thanh ghi (nhớ) / dây tổ hợp
`always @(posedge clk)`	Hành vi tuần tự theo clock
PE	Một ô MAC + vài thanh ghi
Testbench	Môi trường kích thích + kiểm tra đúng sai
VCD + GTKWave	Sóng tín hiệu theo thời gian
Icarus (`iverilog`/`vvp`)	Bộ mô phỏng miễn phí

10.3. Chu trình làm việc Module 3 (toàn bộ do P4)

`RTL` $\rightarrow$ `Testbench + ref` $\rightarrow$ `iverilog` $\rightarrow$ `GTKWave` $\rightarrow$ `đếm chu kỳ / CSV`

Tổng hợp Vivado (LUT/DSP/BRAM) là điểm cộng: ước lượng "to" cỡ nào trên FPGA, chưa bắt buộc lên board thật.

11. Bốn câu hỏi nghiên cứu gắn với lý thuyết

Câu 1. CPU: Với ablation và (10), mỗi kỹ thuật (cache / SIMD / đa lõi) đóng góp bao nhiêu?

Câu 2. GPU: $N^\star$ theo (15) khi so T_{E2E} và khi chỉ so T_{kernel}?

Câu 3. Systolic: Vì sao array đơn giản đạt thông lượng cao (data reuse)? $C(N)$ biến thiên thế nào theo (23)?

Câu 4. Đánh đổi: Linh hoạt $\leftrightarrow$ hiệu suất giữa ba cột ở mục 9, dựa trên metric mục 7.

12. Lộ trình đọc theo vai

Vai	Đọc kỹ	Có thể lướt
P1 Phần I (mở đầu + CPU nền)	§2–4, §7	GPU, systolic sâu
P2 Phần II (CPU tối ưu, Câu 1)	§2–5, §7	Verilog chi tiết
P3 Phần III (GPU, Câu 2)	§2, 3, 6, 7	tiling CPU sâu
P4 Phần IV (systolic, Câu 3)	§2, 7–10	OpenMP sâu
P5 Phần V (đánh đổi, Câu 4)	§7, 9, 11	chi tiết RTL

13. Bảng thuật ngữ (glossary)

GEMM GEneral Matrix Multiply.

FMA Fused Multiply-Add: $a \cdot b + c$.

H2D / D2H Host to Device / Device to Host.

Peak / Sustained
Trần lý thuyết / hiệu năng duy trì thực tế.

Ablation Bật/tắt từng kỹ thuật để đo đóng góp.

$N^\star$ Điểm hòa vốn CPU–GPU (xem (15)).

Dataflow Ai đứng yên, ai chảy (weight-/output-/input-stationary).

Utilization U
Tỷ lệ phần cứng làm việc hữu ích (16).

Roofline Đồ thị gắn GFLOPS với I (5)–(6).

14. Sai lầm thường gặp

1. So mili-giây mô phỏng systolic với mili-giây GPU như cùng một thước đo.
2. Chỉ đo T_{kernel} rồi kết luận "GPU luôn thắng".
3. Tối ưu CPU một phát (OpenMP + tile + flag) rồi không giải thích được Câu 1.
4. Thiết kế RTL quá tham (DMA, nhiều dataflow) rồi không kịp waveform.
5. Đổi máy/Colab mà không ghi lại thông tin máy (file env) — mất khả năng tái lập.

15. Tài liệu tham khảo gợi ý

1. J. L. Hennessy, D. A. Patterson, *Computer Architecture: A Quantitative Approach* — memory hierarchy, parallelism.
2. NVIDIA, *CUDA C Programming Guide* — bộ nhớ GPU, đo thời gian.
3. N. P. Jouppi et al., bài báo Google TPU — dataflow và tinh thần weight-stationary.
4. Slide / bài giảng MI4344 của lớp — ưu tiên thuật ngữ giảng viên dùng.

Lưu ý quan trọng

Khi viết báo cáo chính thức: diễn đạt bằng lời của nhóm. Tài liệu này chỉ là nền tảng nội bộ để "không còn mù mờ".

Hết tài liệu lý thuyết cần thiết — phiên bản đồ án MI4344